

Конспект лекции 12: «Матрица». Начало

Преподаватель: Сергей Шершаков

1. Постановка задачи проектирования класса матрицы

Цель лекции — начать проектирование полноценного, безопасного и эффективного класса для работы с двумерными математическими матрицами в стиле ООП на C++.

- **Компактное представление в памяти:** Чтобы минимизировать накладные расходы (оверхед) и избежать фрагментации памяти, данные матрицы хранятся в виде одномерного динамического массива в куче (динамической памяти), а не в виде массива указателей на массивы.
- **Линейный сдвиг (Offset):** Для обращения к элементу по координатам (i, j) вычисляется линейный индекс в одномерном массиве. Формула пересчета: $index = i \times colsNum + j$.

2. Управление ресурсами и концепция RAII

- Динамическая память — это внешний ресурс, управление которым программист должен взять на себя. При выделении памяти в конструкторе её необходимо гарантированно освободить.
- **Принцип дополняемости:** Для каждого вызова оператора `new[]` в коде программы обязательно должен существовать симметричный и комплиментарный вызов оператора `delete[]`.

3. Конструкторы класса Matrix

Класс может иметь несколько конструкторов, перегружающих друг друга в зависимости от параметров:

- **Конструктор по умолчанию:** Инициализирует матрицу нулевого размера, выставляя указатель на данные в `nullptr`.

```
Matrix::Matrix()  
    : _data(nullptr), _rowsNum(0), _colsNum(0) {}
```

- **Конструктор инициализации:** Принимает количество строк и столбцов, выделяет необходимый объем памяти в куче.

```
Matrix::Matrix(size_t rowsNum, size_t colsNum)  
    : _data(new double[rowsNum * colsNum]()) // () обнуляет элементы
```

```
, _rowsNum(rowsNum)  
, _colsNum(colsNum) {}
```

4. Деструктор класса

- **Определение:** Деструктор — это специальный метод класса, который автоматически вызывается компилятором «за мгновение» до того, как объект будет уничтожен и память, занимаемая его полями, будет освобождена.
- **Синтаксис:** Имя деструктора совпадает с именем класса, но предваряется знаком тильды (~). Деструктор не принимает параметров и не возвращает никаких значений.
- **Количество:** В отличие от конструкторов, у класса может быть **ровно один деструктор**. Его невозможно перегрузить.

```
Matrix::~Matrix() {  
    delete[] _data; // Освобождаем выделенный динамический массив  
}
```